

# 自动化测试系统设计参考文档

## 系统概述

自动化测试系统是一个完整的GUI自动化测试解决方案，包含前端图形化界面、后端API服务和通信模块。

## 系统架构

前端界面(Vue3) ↔ 后端API(FastAPI) ↔ 通信模块 ↔ 被测试机器(Dogtail/PyAutoGUI)

## 前端设计

### 技术栈

- Vue 3 + TypeScript + Element Plus
- Pinia状态管理 + Vue Router路由
- Axios HTTP客户端 + Vite构建工具

### 核心模块

- 操作控制页面:** 连接管理、元素操作、图像操作、拖拽操作
- 仪表板页面:** 系统状态概览、快速操作、统计信息
- 状态管理:** 连接状态、操作历史、配置管理
- API接口层:** 统一的HTTP请求封装

### 主要功能

- 图形化操作界面
- 实时连接状态显示

- 操作历史记录
- 系统配置管理

# 后端设计

---

## 技术栈

- Python + FastAPI + Uvicorn
- Pydantic数据验证
- OpenCV图像处理

## 核心功能

1. **连接管理**: 与被测试机器的TCP连接
2. **操作执行**: 调用通信模块执行自动化操作
3. **API服务**: RESTful API接口
4. **错误处理**: 统一的错误处理和响应格式

## API接口

- **POST /connect** - 连接被测试机器
- **POST /click-element** - 点击元素
- **POST /click-image** - 点击图片
- **POST /drag-to** - 拖拽操作
- **POST /input-text** - 文本输入
- **POST /hotkey** - 快捷键操作

## 通信模块设计

---

### 核心组件

1. **TestMachineCommunicator**: TCP通信客户端
2. **Operation**: 高级操作封装

# 通信协议

```
// 命令格式
{
  "action": "操作类型",
  "params": {"参数": "值"},
  "timestamp": 1234567890.123
}

// 响应格式
{
  "success": true,
  "data": {"结果": "数据"},
  "error": null,
  "message": "操作成功"
}
```

## 支持的操作类型

- 元素操作: 点击、右键点击、双击、设置文本
- 图像操作: 查找图像、点击图像
- 鼠标操作: 移动、拖拽、滚动
- 键盘操作: 文本输入、快捷键

## 使用指南

### 环境准备

- Python 3.8+
- Node.js 16+
- 被测测试机器运行自动化服务

### 快速启动

```
# 启动后端
cd backend
python main.py
```

```
# 启动前端
cd frontend
npm install
npm run dev
```

## 基本操作流程

1. 打开前端界面 (http://localhost:3000)
2. 输入被测试机器IP和端口
3. 点击连接按钮
4. 执行各种自动化操作
5. 查看操作历史和结果

## 操作示例

```
// 点击元素
await operationAPI.clickElement("button[0]", ["push button"]);

// 点击图片
await operationAPI.clickImage("/path/to/image.png", 0.8);

// 拖拽操作
await operationAPI.dragTo(100, 100, 200, 200);
```

## 开发指南

### 前端开发

- 使用Vue 3 Composition API
- TypeScript类型安全
- Element Plus组件库
- Pinia状态管理

### 后端开发

- FastAPI异步处理

- Pydantic数据验证
- 统一错误处理
- 详细日志记录

## 通信模块开发

- TCP Socket通信
- JSON命令协议
- 错误重试机制
- 连接状态管理

## 部署指南

---

### 开发环境

```
# 使用启动脚本
./start.sh # Linux/Mac
start.bat  # Windows
```

### 生产环境

```
# 前端构建
npm run build

# 后端部署
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
```

## Docker部署

```
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports: ["3000:80"]
  backend:
```

```
build: ./backend
ports: ["8080:8080"]
```

# 故障排除

## 常见问题

- 连接失败:** 检查网络和端口
- 操作失败:** 检查元素路径和图像文件
- 性能问题:** 优化网络和图像处理

## 调试方法

- 查看浏览器开发者工具
- 检查后端日志文件
- 使用API测试脚本

## 总结

系统采用前后端分离架构，提供完整的GUI自动化测试解决方案。通过模块化设计，系统具有良好的可扩展性和维护性，支持多种自动化操作类型，满足QGIS等桌面应用的自动化测试需求。